

Práctica de Seminario 3

Libro de Laboratorios

April 16, 2026

Acerca de este documento

Este documento se genero a partir de fuentes en LaTeX en <https://github.com/bootlin/training-materials>.



Correcciones, sugerencias, contribuciones son bienvenidas!

Configuración del entorno para Sunde Diana

Crear la estructura de directorios utilizados en los laboratorios prácticos

Luego de este laboratorio, ud podrá:

- Crear el entorno de desarrollo de todos los laboratorios
- Crear un shell script para inicializar las variables de entorno requeridas para los próximos laboratorios

Todos los laboratorios se realizaron utilizando la versión 22.04 de Ubuntu. Es posible que en una versión anterior o posterior se necesiten instalar distintos paquetes o cambien algunas de las configuraciones. Lo mismo aplica a distribuciones diferentes de GNU/Linux.

Configuración del espacio de trabajo del proyecto

Cree el espacio de trabajo, a través de los siguientes comandos basados en 'Building embedded Linux Systems' (K. Yaghmour):

```
host$ mkdir -p dev_sunde_diana_glibc_labs/{bootloader/data,build-tools/data,buildroot/data,\
download,doc,images,kernel/data,project,rootfs,sysapps/data,tmp,tools,bin}
```

Usando el editor de textos de su preferencia, cree el archivo `dev_sunde_diana_glibc` dentro del directorio `dev_sunde_diana_glibc_labs` con el siguiente contenido:

```
export PROJECT_ROOT=$(pwd)
export ARCH=arm
export TARGET=arm-sunde_diana-linux-gnueabihf
export CROSS_COMPILE=${TARGET}-
export PREFIX=${PROJECT_ROOT}/tools
export TARGET_PREFIX=${PREFIX}/${TARGET}
export PATH=${TARGET_PREFIX}/bin:${PROJECT_ROOT}/bin:${PATH}
export PS1="\`echo $PS1 | rev | cut -c 3- | rev`(diana)\$ "
```

Luego, podemos cargar las variable de entorno necesarias para los labs, ejecutando el script anterior a través del comando `source`:

```
host$ source dev_sunde_diana_glibc
```

o de forma equivalente:

```
host$ . dev_sunde_diana_glibc
```

Asegurese que los parámetros se asignaron de forma correcta!!! (echo `$PROJECT_ROOT`)

De ahora en adelante, antes de comenzar a trabajar en los laboratorios, puede iniciar las variables de entorno necesarias ejecutando el shell script `dev_sunde_diana_glibc`.

Más pautas

Pueden ser útiles en cualquiera de los laboratorios.

- Lea las instrucciones y consejos cuidadosamente. Muchas personas cometen errores o pierden tiempo porque se han perdido una explicación o una guía.
- Lea siempre cuidadosamente los mensajes de error, en particular el primero que se emite. Algunas personas tropiezan con errores muy simples solamente porque especifican una ruta de archivo incorrecta y no prestaron suficiente atención al mensaje de error correspondiente.
- Nunca te quedes atascado con un problema extraño más de 5 minutos. Muestra tu problema a tus colegas o al instructor.
- Solo debe usar el usuario **root** para las operaciones que requieren privilegios de superusuario, tales como: montar un sistema de archivos, cargar un módulo del núcleo, cambiar la propiedad del archivo, configurar la red. La mayoría de las tareas regulares (como descargar, extraer fuentes, compilar ...) se pueden realizar como usuarios regulares. Los comandos en donde se necesite ser usuario **root** están explicitados en las guías anteponiendo el comando **sudo**.
- Si ejecutó comandos desde un shell raíz por error, es posible que su usuario habitual ya no pueda manejar los archivos generados correspondientes. En este caso, use el comando **chown -R myuser.myuser linux-3.4**
Ejemplo: **chown -R myuser.myuser linux-3.4**

Construir un juego de herramientas de compilación cruzada

Objetivo: Aprender a compilar un juego de herramientas de compilación cruzada con la biblioteca glibc para Sunde Diana

Luego de este laboratorio, ud podra:

- Configurar la herramienta *Crosstool-ng*
- Ejecutar *Crosstool-ng* y construir su propio compilador cruzado

Prerequisitos:

- Laboratorio "Configuración del entorno para Sunde Diana"

Instalación de los paquetes necesarios

Instale los paquetes necesarios para este laboratorio:

```
host$ sudo apt-get -y install autoconf flex texinfo unzip \
    help2man libtool-bin libncurses5-dev bison tree \
    git gawk make g++ bzip2 rsync
host$ sudo apt-get clean
```

Nota: tenga en cuenta que estos paquetes pueden estar instalados de algún laboratorio anterior.

Instalación de Qemu

Qemu está disponible en la mayoría de las distribuciones de GNU/Linux. Para instalar qemu ejecutamos:

```
host$ sudo apt-get install qemu-system-arm qemu-user qemu-user-static
```

Vamos a verificar que la version de Qemu sea menor o igual a **v7.2.8** o mayor o igual a **v8.0.0** dado que en las versiones intermedias no funciona el guardado en memoria flash.

Es posible listar las maquinas ARM soportadas por qemu desde la línea de comandos:

```
host$ qemu-system-arm --machine help
```

```
1 Supported machines are:
2 akita                Sharp SL-C1000 (Akita) PDA (PXA270)
3 ast2500-evb          Aspeed AST2500 EVB (ARM1176)
4 ast2600-evb          Aspeed AST2600 EVB (Cortex-A7)
5 borzoi               Sharp SL-C3100 (Borzoi) PDA (PXA270)
6 canon-a1100          Canon PowerShot A1100 IS (ARM946)
7 cheetah              Palm Tungsten|E aka. Cheetah PDA (OMAP310)
8 collie               Sharp SL-5500 (Collie) PDA (SA-1110)
9 connex               Gumstix Connex (PXA255)
10 cubieboard           cubietech cubieboard (Cortex-A8)
11 emcraft-sf2          SmartFusion2 SOM kit from Emcraft (M2S010)
12 fp5280g2-bmc         Inspur FP5280G2 BMC (ARM1176)
```

13 fuji-bmc	Facebook Fuji BMC (Cortex-A7)
14 g220a-bmc	Bytedance G220A BMC (ARM1176)
15 highbank	Calxeda Highbank (ECX-1000)
16 imx25-pdk	ARM i.MX25 PDK board (ARM926)
17 integratorcp	ARM Integrator/CP (ARM926EJ-S)
18 kudo-bmc	Kudo BMC (Cortex-A9)
19 kzm	ARM KZM Emulation Baseboard (ARM1136)
20 lm3s6965evb	Stellaris LM3S6965EVB (Cortex-M3)
21 lm3s811evb	Stellaris LM3S811EVB (Cortex-M3)
22 mainstone	Mainstone II (PXA27x)
23 mcimx6ul-evk	Freescale i.MX6UL Evaluation Kit (Cortex-A7)
24 mcimx7d-sabre	Freescale i.MX7 DUAL SABRE (Cortex-A7)
25 microbit	BBC micro:bit (Cortex-M0)
26 midway	Calxeda Midway (ECX-2000)
27 mps2-an385	ARM MPS2 with AN385 FPGA image for Cortex-M3
28 mps2-an386	ARM MPS2 with AN386 FPGA image for Cortex-M4
29 mps2-an500	ARM MPS2 with AN500 FPGA image for Cortex-M7
30 mps2-an505	ARM MPS2 with AN505 FPGA image for Cortex-M33
31 mps2-an511	ARM MPS2 with AN511 DesignStart FPGA image for Cortex-M3
32 mps2-an521	ARM MPS2 with AN521 FPGA image for dual Cortex-M33
33 mps3-an524	ARM MPS3 with AN524 FPGA image for dual Cortex-M33
34 mps3-an547	ARM MPS3 with AN547 FPGA image for Cortex-M55
35 musca-a	ARM Musca-A board (dual Cortex-M33)
36 musca-b1	ARM Musca-B1 board (dual Cortex-M33)
37 musicpal	Marvell 88w8618 / MusicPal (ARM926EJ-S)
38 n800	Nokia N800 tablet aka. RX-34 (OMAP2420)
39 n810	Nokia N810 tablet aka. RX-44 (OMAP2420)
40 netduino2	Netduino 2 Machine (Cortex-M3)
41 netduinoplus2	Netduino Plus 2 Machine (Cortex-M4)
42 none	empty machine
43 npcm750-evb	Nuvoton NPCM750 Evaluation Board (Cortex-A9)
44 nuri	Samsung NURI board (Exynos4210)
45 orangepi-pc	Orange Pi PC (Cortex-A7)
46 palmetto-bmc	OpenPOWER Palmetto BMC (ARM926EJ-S)
47 quanta-gbs-bmc	Quanta GBS (Cortex-A9)
48 quanta-gsj	Quanta GSJ (Cortex-A9)
49 quanta-q711-bmc	Quanta-Q711 BMC (ARM926EJ-S)
50 rainier-bmc	IBM Rainier BMC (Cortex-A7)
51 raspi0	Raspberry Pi Zero (revision 1.2)
52 raspilap	Raspberry Pi A+ (revision 1.1)
53 raspi2b	Raspberry Pi 2B (revision 1.1)
54 realview-eb	ARM RealView Emulation Baseboard (ARM926EJ-S)
55 realview-eb-mpcore	ARM RealView Emulation Baseboard (ARM11MPCore)
56 realview-pb-a8	ARM RealView Platform Baseboard for Cortex-A8
57 realview-pbx-a9	ARM RealView Platform Baseboard Explore for Cortex-A9
58 romulus-bmc	OpenPOWER Romulus BMC (ARM1176)
59 sabrelite	Freescale i.MX6 Quad SABRE Lite Board (Cortex-A9)
60 smdkc210	Samsung SMDKC210 board (Exynos4210)
61 sonorapass-bmc	OCP SonoraPass BMC (ARM1176)
62 spitz	Sharp SL-C3000 (Spitz) PDA (PXA270)
63 stm32vldiscovery	ST STM32VLDISCOVERY (Cortex-M3)
64 supermicrox11-bmc	Supermicro X11 BMC (ARM926EJ-S)

65	<code>swift-bmc</code>	OpenPOWER Swift BMC (ARM1176) (deprecated)
66	<code>sx1</code>	Siemens SX1 (OMAP310) V2
67	<code>sx1-v1</code>	Siemens SX1 (OMAP310) V1
68	<code>tacoma-bmc</code>	OpenPOWER Tacoma BMC (Cortex-A7)
69	<code>terrier</code>	Sharp SL-C3200 (Terrier) PDA (PXA270)
70	<code>tosa</code>	Sharp SL-6000 (Tosa) PDA (PXA255)
71	<code>verdex</code>	Gumstix Verdex (PXA270)
72	<code>versatileab</code>	ARM Versatile/AB (ARM926EJ-S)
73	<code>versatilepb</code>	ARM Versatile/PB (ARM926EJ-S)
74	<code>vexpress-a15</code>	ARM Versatile Express for Cortex-A15
75	<code>vexpress-a9</code>	ARM Versatile Express for Cortex-A9
76	<code>virt-2.10</code>	QEMU 2.10 ARM Virtual Machine
77	<code>virt-2.11</code>	QEMU 2.11 ARM Virtual Machine
78	<code>virt-2.12</code>	QEMU 2.12 ARM Virtual Machine
79	<code>virt-2.6</code>	QEMU 2.6 ARM Virtual Machine
80	<code>virt-2.7</code>	QEMU 2.7 ARM Virtual Machine
81	<code>virt-2.8</code>	QEMU 2.8 ARM Virtual Machine
82	<code>virt-2.9</code>	QEMU 2.9 ARM Virtual Machine
83	<code>virt-3.0</code>	QEMU 3.0 ARM Virtual Machine
84	<code>virt-3.1</code>	QEMU 3.1 ARM Virtual Machine
85	<code>virt-4.0</code>	QEMU 4.0 ARM Virtual Machine
86	<code>virt-4.1</code>	QEMU 4.1 ARM Virtual Machine
87	<code>virt-4.2</code>	QEMU 4.2 ARM Virtual Machine
88	<code>virt-5.0</code>	QEMU 5.0 ARM Virtual Machine
89	<code>virt-5.1</code>	QEMU 5.1 ARM Virtual Machine
90	<code>virt-5.2</code>	QEMU 5.2 ARM Virtual Machine
91	<code>virt-6.0</code>	QEMU 6.0 ARM Virtual Machine
92	<code>virt-6.1</code>	QEMU 6.1 ARM Virtual Machine
93	<code>virt</code>	QEMU 6.2 ARM Virtual Machine (alias of virt-6.2)
94	<code>virt-6.2</code>	QEMU 6.2 ARM Virtual Machine
95	<code>witherspoon-bmc</code>	OpenPOWER Witherspoon BMC (ARM1176)
96	<code>xilinx-zynq-a9</code>	Xilinx Zynq Platform Baseboard for Cortex-A9
97	<code>z2</code>	Zipit Z2 (PXA27x)

Para los laboratorios vamos a utilizar la plataforma **sunde** versión **diana**. La plataforma Sunde provee de un entorno para desarrollos de diseno SoC. Posee las siguientes características:

- CPU: ARM 400MHz
- Arquitectura: ARM
- Procesador: ARM
- RAM: 128MB
- Flash Nand: 128MB
- USB: 2
- Ethernet: 100Mbits

Obteniendo Crosstool-ng

Vamos al directorio `build-tools`.

Para este laboratorio vamos a utilizar la última versión estable de Crosstool-ng, la cual descargaremos utilizando git:

```
host$ git clone https://github.com/crosstool-ng/crosstool-ng.git
host$ cd crosstool-ng/
host$ git checkout crosstool-ng-1.28.0
```

Instalando Crosstool-ng

Podemos instalar Crosstool-ng de forma global en el sistema o mantenerlo local en su propio directorio. Vamos a elegir la última opción como solución, hacemos:

```
host$ ./bootstrap
host$ ./configure --prefix=$(pwd)
host$ make && make install
```

Luego, es posible obtener la ayuda de Crosstool-ng ejecutando:

```
host$ ./ct-ng help
```

Configuración del juego de herramientas a producir

Una sola instalación de Crosstool-ng permite producir tantos juegos de herramientas como queramos, para diferentes arquitecturas, con diferentes bibliotecas C y diferentes versiones de varios componentes.

Crosstool-ng viene con un juego de archivos de configuración listos para usar de varias configuraciones típicas: Crosstool-ng las llama *samples*. Se pueden listar utilizando `./ct-ng list-samples`.

Vamos a utilizar la muestra `arm-cortex_a8-linux-gnueabi`, que genera un juego de herramientas para el modelo Sunde Diana. La misma puede ser cargada ejecutando:

```
host$ ./ct-ng arm-cortex_a8-linux-gnueabi
```

Luego, para personalizar la configuración, podemos utilizar la interfaz `menuconfig`:

```
host$ ./ct-ng menuconfig
```

Vamos a revisar los campos, algunas opciones pueden ya estar asignadas. Nuestra configuración sera:

En Path and misc options:

- Seleccione `Debug crosstool-NG (CT_DEBUG_CT)` y dentro de esa opción seleccione también `Save intermediate steps (CT_DEBUG_CT_SAVE_STEPS)`. Esto nos va a permitir retomar cualquiera de los pasos de compilación en caso de una falla.
- Cambie `Local tarballs directory (CT_LOCAL_TARBALLS_DIR)` a `${PROJECT_ROOT}/download`. Este directorio es donde se van a descargar los distintos componentes a compilar.
- Cambie `Prefix directory (CT_PREFIX_DIR)` a `${PROJECT_ROOT}/tools/${CT_TARGET}`. Este es el directorio en donde el juego de herramientas se va a instalar. El valor de `${CT_TARGET}` lo calcula Crosstool-ng y lo traduce a la tupla del juego de herramientas.
- Deshabilite `Render the toolchain read-only (CT_PREFIX_DIR_RO)` a fin de poder agregar bibliotecas al juego de herramientas en un futuro.
- Asigne a `Number of parallel jobs (CT_PARALLEL_JOBS)` el doble de la cantidad de núcleos que posee la estación de trabajo (es posible consultar dicha cantidad ejecutando en una terminal: `cat /proc/cpuinfo | grep processor | wc -l`). El valor por defecto de trabajos en paralelo es cero, el cual instruye a Crosstool-ng a asignar la cantidad de procesadores que posee el host.

- Es posible cambiar `Maximum log level to see` que por defecto está en `EXTRA` (`CT_LOG_EXTRA`) a `DEBUG` (`CT_LOG_DEBUG`) para tener más detalles de que es lo que esta pasando durante la construcción en caso de que algo vaya mal. Esta opción hace que la generación sea mas lenta. Lo cambiaremos a `INFO` (`CT_LOG_INFO`) para información sobre el avance, pero sin demasiado detalle.

En `Target options`:

- En `Use specific FPU` (`CT_ARCH_FPU`) completamos con el valor `neon`.
- Seleccionamos `hardware` (FPU) en `Floating point` (`CT_ARCH_FLOAT_HW`).

En `Toolchain options`:

- En `Toolchain ID string` (`CT_TOOLCHAIN_PKGVERSION`) escribimos `Linux Embebido`.
- En `Tuple's vendor string` (`CT_TARGET_VENDOR`) escribimos `sunde_diana`.
- Escriba en `Tuple's alias` (`CT_TARGET_ALIAS`) la cadena `arm-linux`. De esta forma, vamos a poder utilizar el compilador como `arm-linux-gcc` en lugar de `arm-sunde_diana-linux-gnueabi-hf-gcc`, que es mucho más largo de escribir.

En `Debug facilities`:

- Eliminamos la opción `duma` (`CT_DEBUG_DUMA`)

En `Companion libraries`:

- Seleccionamos `Check the companion libraries builds` (`CT_COMPLIBS_CHECK`)
- Cambiamos a la version 0.26 la biblioteca `ISL` (`ISL_V_0_26`)

```
-- isl --->
...
Version of isl (0.26) --->
```

Explore las diferentes opciones disponibles navegando a través de los menus y consultando la ayuda para algunas opciones. No dude en consultar a su instructor por detalles disponibles en las opciones. Sin embargo, recuerde las pruebas de los laboratorios se realizaron con las configuraciones descriptas anteriormente. Puede perder tiempo con problemas inesperados si personaliza la configuración del juego de herramientas.

Ahora ejecutamos:

```
host$ ./ct-ng build
```

y esperamos un rato!

La salida será algo similar a:

```
1 [INFO ] Performing some trivial sanity checks
2 [WARN ] Number of open files 1024 may not be sufficient to build the toolchain; increasing to 2
3 [INFO ] Build started 20260318.162520
4 [INFO ] Building environment variables
5 [INFO ] =====
6 [INFO ] Retrieving needed toolchain components' tarballs
7 [INFO ] Retrieving needed toolchain components' tarballs: done in 0.62s (at 00:07)
8 [INFO ] =====
9 [INFO ] Extracting and patching toolchain components
10 [INFO ] Extracting and patching toolchain components: done in 0.62s (at 00:08)
11 [INFO ] Saving state to restart at step 'companion_tools_for_build'...
12 [INFO ] Saving state to restart at step 'companion_libs_for_build'...
```

```
13 [INFO ] =====
14 [INFO ] Installing ncurses for build
15 [INFO ] Installing ncurses for build: done in 16.61s (at 00:25)
16 [INFO ] Saving state to restart at step 'binutils_for_build'...
17 [INFO ] Saving state to restart at step 'companion_tools_for_host'...
18 [INFO ] Saving state to restart at step 'companion_libs_for_host'...
19 [INFO ] =====
20 [INFO ] Installing zlib for host
21 [INFO ] Installing zlib for host: done in 0.64s (at 00:25)
22 [INFO ] =====
23 [INFO ] Installing zstd for host
24 [INFO ] Installing zstd for host: done in 5.57s (at 00:31)
25 [INFO ] =====
26 [INFO ] Installing GMP for host
27 [INFO ] Installing GMP for host: done in 27.64s (at 00:59)
28 [INFO ] =====
29 [INFO ] Installing MPFR for host
30 [INFO ] Installing MPFR for host: done in 15.61s (at 01:14)
31 [INFO ] =====
32 [INFO ] Installing ISL for host
33 [INFO ] Installing ISL for host: done in 20.59s (at 01:35)
34 [INFO ] =====
35 [INFO ] Installing MPC for host
36 [INFO ] Installing MPC for host: done in 9.70s (at 01:45)
37 [INFO ] =====
38 [INFO ] Installing expat for host
39 [INFO ] Installing expat for host: done in 5.27s (at 01:50)
40 [INFO ] =====
41 [INFO ] Installing ncurses for host
42 [INFO ] Installing ncurses for host: done in 16.45s (at 02:06)
43 [INFO ] =====
44 [INFO ] Installing libiconv for host
45 [INFO ] Installing libiconv for host: done in 0.01s (at 02:06)
46 [INFO ] =====
47 [INFO ] Installing gettext for host
48 [INFO ] Installing gettext for host: done in 0.01s (at 02:06)
49 [INFO ] Saving state to restart at step 'binutils_for_host'...
50 [INFO ] =====
51 [INFO ] Installing binutils for host
52 [INFO ] Installing binutils for host: done in 33.48s (at 02:41)
53 [INFO ] Saving state to restart at step 'linker'...
54 [INFO ] Saving state to restart at step 'libc_headers'...
55 [INFO ] Saving state to restart at step 'kernel_headers'...
56 [INFO ] =====
57 [INFO ] Installing kernel headers
58 [INFO ] Installing kernel headers: done in 6.47s (at 02:56)
59 [INFO ] Saving state to restart at step 'cc_core'...
60 [INFO ] =====
61 [INFO ] Installing core C gcc compiler
62 [INFO ] Installing core C gcc compiler: done in 150.70s (at 05:29)
63 [INFO ] Saving state to restart at step 'libc_main'...
64 [INFO ] =====
```

```
65 [INFO ] Installing C library
66 [INFO ] =====
67 [INFO ] Building for multilib 1/1: ''
68 [INFO ] Building for multilib 1/1: '': done in 79.18s (at 07:12)
69 [INFO ] Installing C library: done in 79.26s (at 07:12)
70 [INFO ] Saving state to restart at step 'cc_for_build'...
71 [INFO ] Saving state to restart at step 'cc_for_host'...
72 [INFO ] =====
73 [INFO ] Installing final gcc compiler
74 [INFO ] Installing final gcc compiler: done in 213.18s (at 11:36)
75 [INFO ] Saving state to restart at step 'libc_post_cc'...
76 [INFO ] Saving state to restart at step 'companion_libs_for_target'...
77 [INFO ] =====
78 [INFO ] Installing GMP for target
79 [INFO ] Installing GMP for target: done in 16.58s (at 13:38)
80 [INFO ] =====
81 [INFO ] Installing MPFR for target
82 [INFO ] Installing MPFR for target: done in 7.55s (at 13:46)
83 [INFO ] =====
84 [INFO ] Installing libelf for the target
85 [INFO ] Installing libelf for the target: done in 3.23s (at 13:49)
86 [INFO ] =====
87 [INFO ] Installing expat for target
88 [INFO ] Installing expat for target: done in 5.04s (at 13:54)
89 [INFO ] =====
90 [INFO ] Installing ncurses for target
91 [INFO ] Installing ncurses for target: done in 21.95s (at 14:16)
92 [INFO ] Saving state to restart at step 'binutils_for_target'...
93 [INFO ] Saving state to restart at step 'debug'...
94 [INFO ] =====
95 [INFO ] Installing cross-gdb
96 [INFO ] Installing cross-gdb: done in 72.81s (at 17:14)
97 [INFO ] =====
98 [INFO ] Installing native gdb
99 [INFO ] Installing native gdb: done in 68.23s (at 18:22)
100 [INFO ] =====
101 [INFO ] Installing gdb server
102 [INFO ] Installing gdb server: done in 25.85s (at 18:48)
103 [INFO ] =====
104 [INFO ] Installing ltrace
105 [INFO ] Installing ltrace: done in 13.68s (at 19:02)
106 [INFO ] =====
107 [INFO ] Installing strace
108 [INFO ] Installing strace: done in 50.18s (at 19:52)
109 [INFO ] Saving state to restart at step 'test_suite'...
110 [INFO ] Saving state to restart at step 'finish'...
111 [INFO ] =====
112 [INFO ] Finalizing the toolchain's directory
113 [INFO ] Stripping all toolchain executables
114 [INFO ] Finalizing the toolchain's directory: done in 3.33s (at 21:50)
115 [INFO ] Build completed at 20260318.164709
116 [INFO ] (elapsed: 21:49.15)
```

117 [INFO] Finishing installation (may take a few seconds)...

Estando dentro del directorio `${PROJECT_ROOT}`, podemos verificar la instalación con el siguiente comando:

```
host$ tree -L 3 --charset=ascii $PROJECT_ROOT/tools/
```

que producirá una salida similar a:

```
1 /home/user/dev/linux-embedded/dev_sunde_diana_glibc_labs/tools/
2 |-- arm-sunde_diana-linux-gnueabi
3 |   |-- arm-sunde_diana-linux-gnueabi
4 |     |-- bin
5 |     |-- debug-root
6 |     |-- include
7 |     |-- lib
8 |     |-- sysroot
9 |   |-- bin
10 |     |-- arm-linux-addr2line -> arm-sunde_diana-linux-gnueabi-addr2line
11 |     |-- arm-linux-ar -> arm-sunde_diana-linux-gnueabi-ar
12 |     |-- arm-linux-as -> arm-sunde_diana-linux-gnueabi-as
13 |     |-- arm-linux-c++ -> arm-sunde_diana-linux-gnueabi-c++
14 |     |-- arm-linux-cc -> arm-sunde_diana-linux-gnueabi-cc
15 |     |-- arm-linux-c++filt -> arm-sunde_diana-linux-gnueabi-c++filt
16 |     |-- arm-linux-cpp -> arm-sunde_diana-linux-gnueabi-cpp
17 |     |-- arm-linux-elfedit -> arm-sunde_diana-linux-gnueabi-elfedit
18 |     |-- arm-linux-g++ -> arm-sunde_diana-linux-gnueabi-g++
19 |     |-- arm-linux-gcc -> arm-sunde_diana-linux-gnueabi-gcc
20 |     |-- arm-linux-gcc-15.2.0 -> arm-sunde_diana-linux-gnueabi-gcc-15.2.0
21 |     |-- arm-linux-gcc-ar -> arm-sunde_diana-linux-gnueabi-gcc-ar
22 |     |-- arm-linux-gcc-nm -> arm-sunde_diana-linux-gnueabi-gcc-nm
23 |     |-- arm-linux-gcc-ranlib -> arm-sunde_diana-linux-gnueabi-gcc-ranlib
24 |     |-- arm-linux-gcov -> arm-sunde_diana-linux-gnueabi-gcov
25 |     |-- arm-linux-gcov-dump -> arm-sunde_diana-linux-gnueabi-gcov-dump
26 |     |-- arm-linux-gcov-tool -> arm-sunde_diana-linux-gnueabi-gcov-tool
27 |     |-- arm-linux-gdb -> arm-sunde_diana-linux-gnueabi-gdb
28 |     |-- arm-linux-gdb-add-index -> arm-sunde_diana-linux-gnueabi-gdb-add-index
29 |     |-- arm-linux-gprof -> arm-sunde_diana-linux-gnueabi-gprof
30 |     |-- arm-linux-gstack -> arm-sunde_diana-linux-gnueabi-gstack
31 |     |-- arm-linux-ld -> arm-sunde_diana-linux-gnueabi-ld
32 |     |-- arm-linux-ld.bfd -> arm-sunde_diana-linux-gnueabi-ld.bfd
33 |     |-- arm-linux-ldd -> arm-sunde_diana-linux-gnueabi-ldd
34 |     |-- arm-linux-lto-dump -> arm-sunde_diana-linux-gnueabi-lto-dump
35 |     |-- arm-linux-nm -> arm-sunde_diana-linux-gnueabi-nm
36 |     |-- arm-linux-objcopy -> arm-sunde_diana-linux-gnueabi-objcopy
37 |     |-- arm-linux-objdump -> arm-sunde_diana-linux-gnueabi-objdump
38 |     |-- arm-linux-populate -> arm-sunde_diana-linux-gnueabi-populate
39 |     |-- arm-linux-ranlib -> arm-sunde_diana-linux-gnueabi-ranlib
40 |     |-- arm-linux-readelf -> arm-sunde_diana-linux-gnueabi-readelf
41 |     |-- arm-linux-size -> arm-sunde_diana-linux-gnueabi-size
42 |     |-- arm-linux-strings -> arm-sunde_diana-linux-gnueabi-strings
43 |     |-- arm-linux-strip -> arm-sunde_diana-linux-gnueabi-strip
44 |     |-- arm-sunde_diana-linux-gnueabi-addr2line
```

```

45 | |-- arm-sunde_diana-linux-gnueabi-hf-ar
46 | |-- arm-sunde_diana-linux-gnueabi-hf-as
47 | |-- arm-sunde_diana-linux-gnueabi-hf-c++
48 | |-- arm-sunde_diana-linux-gnueabi-hf-cc -> arm-sunde_diana-linux-gnueabi-hf-gcc
49 | |-- arm-sunde_diana-linux-gnueabi-hf-c++filt
50 | |-- arm-sunde_diana-linux-gnueabi-hf-cpp
51 | |-- arm-sunde_diana-linux-gnueabi-hf-elfedit
52 | |-- arm-sunde_diana-linux-gnueabi-hf-g++
53 | |-- arm-sunde_diana-linux-gnueabi-hf-gcc
54 | |-- arm-sunde_diana-linux-gnueabi-hf-gcc-15.2.0
55 | |-- arm-sunde_diana-linux-gnueabi-hf-gcc-ar
56 | |-- arm-sunde_diana-linux-gnueabi-hf-gcc-nm
57 | |-- arm-sunde_diana-linux-gnueabi-hf-gcc-ranlib
58 | |-- arm-sunde_diana-linux-gnueabi-hf-gcov
59 | |-- arm-sunde_diana-linux-gnueabi-hf-gcov-dump
60 | |-- arm-sunde_diana-linux-gnueabi-hf-gcov-tool
61 | |-- arm-sunde_diana-linux-gnueabi-hf-gdb
62 | |-- arm-sunde_diana-linux-gnueabi-hf-gdb-add-index
63 | |-- arm-sunde_diana-linux-gnueabi-hf-gprof
64 | |-- arm-sunde_diana-linux-gnueabi-hf-gstack
65 | |-- arm-sunde_diana-linux-gnueabi-hf-ld
66 | |-- arm-sunde_diana-linux-gnueabi-hf-ld.bfd
67 | |-- arm-sunde_diana-linux-gnueabi-hf-ldd
68 | |-- arm-sunde_diana-linux-gnueabi-hf-lto-dump
69 | |-- arm-sunde_diana-linux-gnueabi-hf-nm
70 | |-- arm-sunde_diana-linux-gnueabi-hf-objcopy
71 | |-- arm-sunde_diana-linux-gnueabi-hf-objdump
72 | |-- arm-sunde_diana-linux-gnueabi-hf-populate
73 | |-- arm-sunde_diana-linux-gnueabi-hf-ranlib
74 | |-- arm-sunde_diana-linux-gnueabi-hf-readelf
75 | |-- arm-sunde_diana-linux-gnueabi-hf-size
76 | |-- arm-sunde_diana-linux-gnueabi-hf-strings
77 | |-- arm-sunde_diana-linux-gnueabi-hf-strip
78 |-- build.log.bz2
79 |-- include
80 |-- lib
81 | |-- bfd-plugins
82 | |-- gcc
83 | |-- libgcc1.so -> libgcc1.so.0.0.0
84 | |-- libgcc1.so.0 -> libgcc1.so.0.0.0
85 | |-- libgcc1.so.0.0.0
86 |-- libexec
87 | |-- gcc
88 |-- share
89 | |-- arm-sunde_diana-linux-gnueabi-hf-ct-ng.config.bz2
90 | |-- gcc-15.2.0
91 | |-- gdb
92 | |-- licenses
93
94 18 directories, 73 files

```

Si hay algún problema en la compilación, podemos retornar al último paso exitoso ejecutando:

```
host$ ./ct-ng <step>+
```

Donde la lista de pasos se puede conocer con el comando:

```
host$ ./ct-ng list-steps
```

Por ejemplo, si falla en el paso *'Installing pass-2 core C gcc compiler'*, podemos reestablecer desde el estado anterior con el siguiente comando:

```
host$ ./ct-ng cc_core_pass_2+
```

Problemas conocidos

Los archivos fuente no se encuentran en Internet

Es frecuente que Crosstool-ng falle debido a que no puede encontrar algunos archivos fuente en Internet, cuando dicho archivo fue movido o reemplazado por una versión mas reciente. Las nuevas versiones de Crosstool-ng vienen con URLs actualizadas, pero mientras tanto, es necesario una solución.

Si se encuentra con este problema, lo que puede hacer es buscar el archivo por su cuenta en Internet, y copiarlo al directorio `${PROJECT_ROOT}/download`. Tenga en cuenta que los archivos fuente pueden estar comprimidos de diferentes formas (por ejemplo, terminar con `.gz` en lugar de `.bz2`), en cualquiera de los casos esta bien. Luego, todo lo que tiene que hacer es correr nuevamente `./ct-ng build`, y el mismo va a utilizar los fuentes que descargo.

Error: "Don't set LD_LIBRARY_PATH. It screws up the build"

Si nos aparece el siguiente error:

```
[INFO ] Performing some trivial sanity checks
[ERROR] Don't set LD_LIBRARY_PATH. It screws up the build.
[ERROR]
...
[ERROR]
[ERROR] (elapsed: 26365905:15.92)
[00:00] / ct-ng:261: recipe for target 'build' failed
make: *** [build] Error 1
```

es porque tenemos asignada la variable `LD_LIBRARY_PATH`, la cual tiene conflictos con Crosstool-ng.

Para solucionar el problema debemos limpiar la variable de entorno:

```
host$ unset LD_LIBRARY_PATH
```

Probando el juego de herramientas

Para probar el juego de herramientas vamos a utilizar el directorio `${PROJECT_ROOT}/project`. Ahi dentro vamos a crear nuestro programa `hello.c` el cual vamos a compilar con el nuevo juego de herramientas de compilación cruzada. También vamos a utilizar la herramienta `make` para automatizar el proceso.

Para compilar el programa de ejemplo `hello.c` de forma *estática* ejecutamos el comando `arm-sunde_diana-linux-gnueabi-hf-gcc`.

```
host$ arm-sunde_diana-linux-gnueabi-hf-gcc -static hello.c -o hello
```

o utilizando el enlace `arm-linux-gcc` si escribió un alias al configurar el juego de herramientas (`CT_TARGET_ALIAS`):

```
host$ arm-linux-gcc -static hello.c -o hello
```

hello.c:

```
#include <stdio.h>
#include <stdlib.h>

int main (void)
{
    printf("Hola Mundo!\n");
    return EXIT_SUCCESS;
}
```

Es posible utilizar el comando `file` sobre el binario para asegurarse que fue compilado correctamente para la arquitectura ARM.

Luego, podemos utilizar `qemu` para probar el ejecutable:

```
host$ qemu-arm hello
```

Limpiando

Para liberar aproximadamente 7 GB de espacio en disco, realice un `./ct-ng clean` en el directorio fuente de Crosstool-ng. Esto va a remover el código fuente de diferentes componentes del juego de herramientas, como así también los archivos generados que son ahora innecesarios dado que el juego de herramientas fue instalado en `${PROJECT_ROOT}/tools`.

Utilizando Makefile

El archivo `Makefile` es útil para construir aplicaciones en el espacio de usuario; según K. Yaghmour ('Building embedded Linux Systems'):

```
AS          = $(CROSS_COMPILE)as
AR          = $(CROSS_COMPILE)ar
CC          = $(CROSS_COMPILE)gcc
CPP         = $(CC) -E
LD          = $(CROSS_COMPILE)ld
NM          = $(CROSS_COMPILE)nm
OBJCOPY     = $(CROSS_COMPILE)objcopy
OBJDUMP     = $(CROSS_COMPILE)objdump
RANLIB      = $(CROSS_COMPILE)ranlib
READELF     = $(CROSS_COMPILE)readelf
SIZE        = $(CROSS_COMPILE)size
STRINGS     = $(CROSS_COMPILE)strings
STRIP       = $(CROSS_COMPILE)strip

export AS AR CC CPP LD NM OBJCOPY OBJDUMP RANLIB \
        READELF SIZE STRINGS STRIP

CFLAGS      = -O2 -Wall
HEADER_OPS  =
LDFLAGS     =

EXEC_NAME   = hello_command
```

```
INSTALL      = install
INSTALL_DIR  = ${PROJECT_ROOT}/rootfs/bin

CFILES       = hello.c
HFILES       =

OBJS         = $(CFILES:%.c=%.o)

all: hello

.c.o:
    $(CC) $(CFLAGS) $(HEADER_OPS) -c $<

hello: $(OBJS) $(HFILES)
    $(CC) -o $(EXEC_NAME) $^ $(LDFLAGS)

install: hello
    test -d $(INSTALL_DIR) || $(INSTALL) -d -m 755 $(INSTALL_DIR)
    $(INSTALL) -m 755 $(EXEC_NAME) $(INSTALL_DIR)

clean:
    rm -f *.o $(EXEC_NAME)

distclean:
    rm -f *~
    rm -f *.o $(EXEC_NAME)
```


Kernel - Compilación cruzada

Objetivo: Aprender a compilar de forma cruzada el núcleo de Linux para una plataforma ARM.

Luego de este laboratorio, ud podrá:

- Configurar un entorno de compilación cruzada
- Configurar de forma acorde el archivo Makefile del núcleo de Linux
- Compilar de forma cruzada el núcleo de Linux para la placa Versatile Express en el emulador Qemu
- Utilizar U-Boot para descargar el núcleo de Linux
- Verificar que el núcleo de Linux que compilamos inicia el sistema

Prerequisitos:

- Laboratorio "Gestor de arranque - U-Boot"

Dependencias

Vamos a instalar algunas dependencias necesarias al momento de compilar:

```
host$ sudo apt install libgmp3-dev libmpc-dev
```

Configuración

Cambie al directorio `${PROJECT_ROOT}/kernel`.

Sistema destino

Vamos a compilar de forma cruzada e iniciar el núcleo de Linux para la placa `sunde-diana`.

Fuentes del núcleo de Linux

Vamos a descargar los fuentes del núcleo de Linux desde el sitio <https://www.kernel.org/>. Dado que el código fuente del núcleo con su historia es muy grande, vamos a descargar solamente una rama particular. Para ello utilizaremos la herramienta `git` de la siguiente forma:

```
host$ git clone -b v6.16.7 --depth 1 \
    git://git.kernel.org/pub/scm/linux/kernel/git/stable/linux-stable.git
host$ cd linux-stable
```

Configuración del entorno de compilación cruzada

Para compilar el núcleo de Linux para ARCH, necesitamos tener un juego de herramientas de compilación cruzada instalado en el sistema. Vamos a utilizar el juego de herramientas de compilación cruzada que construimos en los laboratorios anteriores.

Parche del Kernel

Dado que en el núcleo no hay soporte para la Sunde Diana, vamos a agregar los archivos necesarios usando un parche. Vamos a copiar el archivo del parche `add_sunde_diana_platform`.

patch a la carpeta `$PROJECT_ROOT/kernel`. Luego para aplicar el parche, estando dentro de los fuentes del núcleo, ejecutamos:

```
host$ patch -p 1 < ../add_sunde_diana_platform.patch
```

Configuración del núcleo de Linux

Para que el núcleo use la configuración de la placa Sunde Diana, ejecutamos `make sunde_diana_defconfig`, esto va a generar el archivo `.config` en la raíz de los fuentes.

Podemos personalizar las nuevas opciones corriendo:

```
host$ make menuconfig
```

En la configuración, como experimento, cambiamos la compresión del núcleo de Gzip a XZ (`CONFIG_KERNEL_XZ`). Este algoritmo de compresión es mucho más eficiente que Gzip, en términos de relación de compresión a expensas de un tiempo de descompresión mayor.

También vamos a habilitar el montaje automático del directorio `/dev` (`CONFIG_DEVTMPFS_MOUNT`):

```
Device Drivers --->
  Generic Driver Options --->
    ...
    [*] Maintain a devtmpfs filesystem to mount at /dev
    [*] Automount devtmpfs at /dev, after the kernel mounted the rootfs
    ...
```

Compilación cruzada

Ahora estamos listos para compilar de forma cruzada el núcleo. Simplemente ejecutamos:

```
host$ make
```

y espere un rato a que el núcleo se compile. No olvide utilizar `make -j<n>` si posee múltiples núcleos en la máquina!

Busque al final de la salida de la construcción del núcleo para ver que archivos contiene la imagen del kernel. También es posible ver los archivos del árbol de dispositivos (Device Tree) `.dtb` que fueron compilados. Encuentre que archivo `.dtb` corresponde a su placa.

Ahora vamos a armar una imagen compatible con el U-Boot que viene por defecto en la placa. Para ello vamos a ejecutar los siguientes pasos:

```
host$ cat ${PROJECT_ROOT}/kernel/linux-stable/arch/arm/boot/zImage \
    ${PROJECT_ROOT}/kernel/linux-stable/arch/arm/boot/dts/nxp/imx/imx53-diana.dtb \
    > zImage-imx53-diana
host$ mkimage -A arm -O linux -T kernel -C none -a 0x70800000 -e 0x70800000 \
    -n "Linux-6.x-imx53-diana" -d zImage-imx53-diana uImage-imx53-diana
```

Pruebas de red

Para probar si funciona la conexión de red entre u-boot y la interfaz de red, debemos primero asignar una dirección de IP a U-Boot:

```
U-Boot => setenv ipaddr 192.168.7.101
```

y luego hacemos un ping:

```
U-Boot => ping 192.168.7.100
```

Si el ping fue exitoso, tendremos una salida similar a:

```
smc911x: detected LAN9118 controller
smc911x: phy initialized
smc911x: MAC 52:54:00:12:34:56
Using ethernet@3,02000000 device
smc911x: MAC 52:54:00:12:34:56
host 192.168.7.100 is alive
```

Configuración de una comunicación Ethernet

Más adelante vamos a transferir archivos desde la estación de trabajo a la placa utilizando el protocolo TFTP, que trabaja sobre la conexión Ethernet.

Para comenzar, instale y configure el servidor TFTP en la estación de trabajo:

```
host$ sudo apt-get install tftpd-hpa
```

Los archivos que queramos compartir luego por tftp deben ubicarse en el directorio `/srv/tftp`.

Para poder probar si el servicio de tftp esta funcional, debemos instalar un cliente tftp. Para ello ejecutamos:

```
host$ sudo apt-get install tftp-hpa
```

Vamos a agregar un archivo de prueba para servir a traves de tftp. Para ello hacemos:

```
host$ echo "Linux Embebido" | sudo tee /srv/tftp/readme.txt > /dev/null
```

Ahora nos movemos al directorio temporal `${PROJECT_ROOT}/tmp`, y allí ejecutamos el cliente tftp intentando descargar algunos de los archivos:

```
host$ tftp 127.0.0.1
tftp> get readme.txt
tftp> q
```

Si funciona la transferencia, en el directorio actual debería estar el archivo `readme.txt`.

Ahora configure la IP del servidor a través de U-Boot asignando la variable de entorno `serverip`:

```
U-Boot => setenv serverip 192.168.7.100
```

Para hacer permanente los cambios, guarde las variables de entorno (recuerde levantar Qemu con el soporte para flash, como vimos en el laboratorio anterior):

```
U-Boot => saveenv
```

Cargue e inicie el kernel usando U-Boot

Necesitamos configurar la variable de entorno `bootargs` para poder ver salida por pantalla. Por lo tanto, hacemos:

```
U-Boot => setenv bootargs 'console=ttymxc0,115200'
```

Vamos a utilizar TFTP para cargar la imagen del Kernel en la memoria de U-Boot.

En su estación de trabajo (usando el comando `sudo`), copiamos el archivo `uImage-imx53-diana` al directorio expuesto por el servidor de TFTP (en general es el directorio `/srv/tftp`, pero depende de la distribución). Puede crear una carpeta con el nombre `diana` para organizar los archivos ahí dentro.

En el destino, cargamos `uImage-imx53-diana` desde TFTP en la RAM en la dirección `0x72000000`:

```
U-Boot => tftp 0x72000000 uImage-imx53-diana
```

Recordemos que en ese archivo esta el núcleo y el dtb combinados.

Inicie el núcleo de Linux:

```
U-Boot => bootm 0x72000000
```

Debería ver como se inicia Linux y hasta que finalmente se bloquea por un Kernel panic, como en el caso anterior.

Ahora es posible automatizar estos pasos para que cada vez que se inicia o reinicia la placa se ejecuten los comandos anteriores. Especifique un `bootcmd` diferente:

```
U-Boot => setenv bootcmd 'tftp 0x72000000 uImage-imx53-diana;  
    bootm 0x72000000'  
U-Boot => saveenv
```

Utilizando un sistema de construcción.

Ejemplo con Buildroot

Objetivo: descubrir como funciona y como utilizar un sistema de construcción, utilizando como ejemplo el sistema de construcción Buildroot. Construir un sistema GNU/Linux con bibliotecas y hacerlo funcionar en la placa.

Configuración

Vamos al directorio `${PROJECT_ROOT}/buildroot`.

Obtenemos Buildroot y exploramos el código fuente

El sitio oficial Buildroot está disponible en <http://buildroot.org/>. Vamos a descargar la última versión utilizando `git`, por lo tanto ejecutamos:

```
host$ git clone https://git.buildroot.net/buildroot
host$ cd buildroot
host$ git checkout 2026.02
```

Existen varios subdirectorios y archivos, de los cuales los más importantes son:

- **boot** contiene los Makefiles y los items de configuración relacionados con la compilación de gestores de arranque comunes (Grub, U-Boot, Barebox, etc.)
- **configs** contiene un juego predefinido de configuraciones, similar al concepto de `defconfig` en el núcleo
- **docs** contiene la documentación de Buildroot. Puede comenzar por leer `buildroot.html` que es la principal documentación de Buildroot
- **fs** contiene el código usado para generar varios de los formatos de imagen del sistema de archivos raíz
- **linux** contiene el Makefile y los items de configuración relacionados con la compilación del núcleo de Linux
- **Makefile** es el principal Makefile que vamos a utilizar para en Buildroot: todo funciona a través de Makefiles en Buildroot;
- **package** es un directorio que contiene todos los Makefiles, parches e items de configuración para compilar las aplicaciones en el espacio de usuario y bibliotecas de su sistema embebido. Heche un vistazo a varios de sus subdirectorio y vea que es lo que contienen;
- **system** contiene el esqueleto del sistema de archivos raíz y las *tablas de dispositivos* utilizadas cuando se usa un directorio `/dev` estático;
- **toolchain** contiene los Makefiles, parches e items de configuración para generar el juego de herramientas de compilación cruzada.

Configurar Buildroot

Vamos a revisar el directorio `configs` y seleccionar la placa que sea similar a la que estamos utilizando, para nuestro caso `mx53loco`:

```
host$ make mx53loco_defconfig
```

Vamos a realizar las siguientes tareas:

- Generar un sistema embebido en Linux para ARM
- Utilizar un juego de herramientas de compilación cruzada existente, en lugar de que Buildroot lo genere por nosotros
- Integrar Busybox en nuestro sistema embebido en Linux
- Integrar el sistema de archivos destino en un empaquetado (tarball)

Para ejecutar la utilizad de configuración de Buildroot, simplemente ejecutamos:

```
host$ make menuconfig
```

Asignamos las siguientes opciones:

- Toolchain
 - Toolchain type: External toolchain
 - Toolchain: Custom toolchain
 - Toolchain path: vamos a utilizar en juego de herramientas que construimos: `${PROJECT_ROOT}/tools/arm-sunde_diana-linux-gnueabi`
 - External toolchain gcc version: 15.x
 - External toolchain kernel headers series: 6.16.x. Indicamos la misma versión que utilizamos para generar el juego de herramientas.
 - External toolchain C library: glibc/eglibc
 - Debemos decirle a Buildroot acerca de la configuración de nuestro juego de herramientas, por lo tanto habilitamos: `Toolchain has SSP support?` y `Toolchain has C++ support?`. Deshabilitamos: `Toolchain has RPC support?` Igualmente Buildroot va a verificar estos parametros.
 - Seleccionamos `Copy gdb server to the Target`
- Kernel
 - Deshabilitamos Linux Kernel
- Target packages
 - Filesystem and flash utilities
 - * nilfs-utils
- Filesystem images
 - Selecciona `tar the root filesystem`
- Bootloaders
 - Deshabilitamos U-Boot

Salimos de la interfaz de `menuconfig`. La configuración quedó guardada en el archivo `.config`.

Generar el sistema GNU/Linux embebido

Simplemente ejecutamos:

```
host$ make
```

Nota: no utilizamos la opción `-j` dado que dicha configuración se realiza a través de la interfaz `menuconfig`.

En primer lugar, Buildroot va a crear un pequeño entorno con el juego de herramientas externo, luego descargará, extraerá, configurará, compilará e instalará cada uno de los componentes del sistema embebido.

La compilación termina con un error dado que no estamos generando el núcleo de Linux y la herramienta quiere copiarlo al final. El error es algo así:

```
host$ ERROR: file(zImage): stat(/home/user/dev/linux-embedded/dev_sunde_diana_glibc_labs/buildroot/output/images/zImage): stat failed
ERROR: vfat(boot.vfat): could not setup zImage
INFO: cmd: "rm -rf "/home/user/dev/linux-embedded/dev_sunde_diana_glibc_labs/buildroot/buildroot/output/images
make[1]: *** [Makefile:834: target-post-image] Error 1
make: *** [Makefile:83: _all] Error 2
```

Toda la compilación se lleva a cabo en el subdirectorio `output/`. Vamos a explorar su contenido:

- **build**, es el directorio en donde cada componente construido por Buildroot es extraído, y donde se lleva a cabo la construcción
- **host**, es el directorio en donde Buildroot instala algunos componentes para el host. Como Buildroot no quiere depender de componentes instalados en la máquina del desarrollador, instala algunas herramientas necesarias para compilar los paquetes para el destino. En nuestro caso instaló herramientas para generar la imagen del sistema de archivos raíz (`genext2fs`, `makedevs`, `fakeroot`)
- **images**, contiene las imágenes finales producidas por Buildroot. En nuestro caso es solo un archivo `.tar` del sistema de archivos llamado `rootfs.tar`, pero dependiendo de la configuración de Buildroot, pueden existir otras imágenes como la del núcleo o la del gestor de arranque.
- **staging**, contiene el espacio de "construcción" del sistema destino. Todas las bibliotecas del destino, con encabezados, y documentación. También contiene los encabezados y la biblioteca C del sistema, que en nuestro caso fue copiada desde el juego de herramientas de compilación cruzada.
- **target**, es el sistema de archivos raíz del destino. Todas las aplicaciones, bibliotecas, en general reducidas (`stripped`) se encuentran instaladas en este directorio. Sin embargo, no puede ser utilizado directamente como sistema de archivos raíz, dado que los archivos de dispositivo no se encuentran: no es posible crearlos sin ser super usuario, y Buildroot tiene una política de no correr como super usuario.

Corriendo el sistema generado

Volvemos al directorio `${PROJECT_ROOT}/buildroot`. Creamos un nuevo directorio `rootfs` que es el que va a contener nuestro sistema de archivos raíz, que va a ser exportado a través de NFS. Vamos a dicho directorio y desempaquetamos el `rootfs` utilizando el siguiente comando:

```
host$ sudo tar -xvf ../buildroot/output/images/rootfs.tar
```

No olvidemos agregar nuestro directorio `rootfs` a la lista de directorios exportados por NFS en el archivo `/etc/exports`, y asegurándonos que el destino lo está utilizando.

Iniciamos la placa, e ingresamos al sistema (cuenta **root**, sin clave).

Ahora deberíamos tener un shell, donde poder correr las herramientas de **nilfs**.

Algunas pruebas más

- Agregamos **dropbear** (el servidor y cliente SSH) a la lista de paquetes a construir por Buildroot e ingresamos en el sistema destino a través de un cliente ssh desde su estación de desarrollo. Consejo: vamos a necesitar asignar una clave no vacía para la cuenta de super usuario en el sistema destino para que esto funcione.